海兰云算力租赁平台
第一阶段技术方案

以 2 台 RTX 5090 起步，60 天交付可对外收费的卡池自助服务

版本 v0.2　2026-11-02　技术方案 / Demo：陆强　评审：曹胜建

0 · 一页结论

我们卖的不是一个"平台"，是一套"能让卡马上租出去并收到钱"的最小闭环。

60 天只做四件事：**卡能分配、租户能隔离、订单能计费、客户能自助。**调度、监控、容器、镜像全部用成熟开源，不自研。

三个必须让客户记住的点：

1. **不做 GPU 切分**——5090 本身不支持 MIG，客户也明确说不需要，这块最贵的成本直接不存在。
2. **60 天出的是能收钱的卡池，不是 PPT 架构**——第 8 周演示"下单→拿卡→跑任务→出账单"完整链路。
3. **第三阶段的 Token 平台不用重做**——账户、额度、计费、模型网关已有在跑的版本，第一阶段建的账户以后直接接着用。

1 · 需求还原

下表是从 10 月 30 日会议纪要中提炼的客户原话与工程含义，用于确认双方理解一致。

客户说的	工程含义	第一阶段怎么做
颗粒度覆盖单卡、单机、多卡、多机	资源分配单元是"卡"，不是虚拟机	单卡 / 整机两种规格先上；多机走同一套编排，不额外开发
不需要做 GPU 微切分	不做 vGPU / MIG / 时间片共享	整卡独占，一张卡同一时刻只属于一个租户
优先基于 K8s 行业标准架构	运维、招人、扩展都按 K8s 生态走	K8s + 标准 Device Plugin，不引入私有调度器
自助下单、资源发放、计费	公有云式自助体验	控制台下单 → 自动创建实例 → 按卡时计费 → 出账
资源自动下发、回收释放	租期结束必须自动收回，不能靠人手动删	实例生命周期状态机 + 到期回收 + 异常兜底
三阶段平滑演进，起步成本优先	架构要留口，但不能一次建完	P1 只建 P1 需要的；对外接口与账户体系按 P3 预留

2 · 交付边界

P1 · 本次交付

卡池租赁平台

单卡 / 整机 / 小集群，自助下单、自动发放、按卡时计费。2 台 5090 起步。

P2 · 后续

多节点分布式

面向大 B 的多机训练任务，RDMA 高速网络、拓扑感知调度、gang scheduling。

P3 · 远期

Token / MaaS

模型广场、推理服务、按 token 计费。账户与计费体系复用 P1。

P1 分层架构



蓝色为需要写的部分，灰色为直接部署的开源组件。三层自研合计约 40–45 人天，是整个 60 天工期的实际工作量。

3 · 模块清单与工作量

模块	做什么	用什么	性质	人天
卡池管理	把节点上的卡登记成池，按单卡 / 整机分配，租期结束自动回收	K8s Device Plugin + 自研资源模型	自研	8–10
租户与配额	开通租户、限制卡数上限、网络与数据隔离	Namespace + ResourceQuota + Kyverno	自研	5
订单与计费	下单、预扣、按分钟计量、结算、出账、对账	自研（复用现有支付幂等模型）	自研	12–15
自助控制台	客户下单页、实例列表、连接入口、账单页	React + 现有组件库	自研	10–12
监控采集	卡利用率、显存、温度、功耗、ECC、掉卡	Prometheus + DCGM Exporter + node_exporter	部署	3
监控大屏	卡池地图、总利用率、今日卡时与收入、告警滚动	Grafana 或自研单页	薄	2
运维管理页	节点管理、实例强制释放、租户封禁、账单导出	并入控制台后台	自研	含上

合计约 40–45 人天。60 个自然日按 1 名主力 + 1 名兼职投入，含 20% 缓冲。

4 · 开源选型

结论先说：**没有一套现成开源能直接当"算力租赁平台"卖。**能直接用开源顶上的是调度、监控、容器、镜像这几层；订单、计费、租户开通必须自己写——这三块恰好是和客户账户体系绑死、也是最愿意付钱的部分。

组件	许可证	在我们方案里的角色	为什么不选替代品
Kubernetes / k3s	Apache-2.0	统一资源底座，客户已认可	OpenStack 体量过大，2 台机器跑不动也用不上
NVIDIA GPU Operator	Apache-2.0	驱动、容器运行时、Device Plugin 一键纳管	手装驱动在 2 台以上就无法维护
Volcano	Apache-2.0	队列配额、gang scheduling，为 P2 多机预留	默认调度器不支持队列与组调度
Prometheus + DCGM Exporter	Apache-2.0	GPU 指标采集与告警的事实标准	自研采集器成本高且拿不到 XID / ECC
Grafana	AGPL-3.0	运维看板；对外大屏建议自研单页	AGPL 只部署不改源码即可，但对外产品化时规避
JupyterHub	BSD-3	租户开箱即用的开发环境	自研 Notebook 编排不划算
Harbor	Apache-2.0	租户私有镜像仓库	—
KubeVirt	Apache-2.0	仅当客户要求"给整机 / 给 Root 虚拟机"时启用	P1 默认给容器，虚拟机按需开启

关于 "开源包一下就能卖"

能做，但要说清包的是哪一层。调度器和监控是"零件"，谁都拿得到，卖不出价；客户付钱买的是**计费准不准、回收干不干净、出问题谁兜**。所以我们的做法是：零件全用开源省时间，把 60 天花在业务层和稳定性上。

5 · 60 天节奏

周次	目标	交付物	验收
W1–W2	底座就位	K8s + GPU Operator 跑通；2 台机器 16 卡全部可识别；Prometheus + DCGM 出图；卡池数据模型定稿	能在大屏上看到每张卡的状态
W3–W4	链路打通	下单 → 创建实例 → SSH / Jupyter 连进去 → 到期回收	手动下单能完整跑一遍，不靠人删资源
W5–W6	能收钱	预扣、按分钟计量、结算、账单页、对账任务	跑 24 小时，账单与真实占用对得上
W7–W8	能演示	控制台打磨、大屏、运维管理页、交付文档、2 台机器压测	客户现场 10 分钟演示不翻车

原则：**先砍功能，不砍时间。**时间一压缩，交付质量就没有下限；功能砍掉只是少一个卖点。

6 · 明确不做的范围

不做的事	原因
GPU 微切分 / vGPU / MIG	5090 硬件不支持 MIG；客户明确不需要；做了也没有收益
RDMA 多机分布式训练调度	属于 P2；P1 只有 2 台机器，做不出真实拓扑
模型推理服务与 token 计费	属于 P3；但账户与计费结构按 P3 预留
AI 自动调优 Agent	见下方专门说明——不是 P1 需求，放进 P1 会挤掉计费工期
平台自身高可用	2 台机器无法做主备；如实告知，不承诺 99.9%

"AI 自动调优 Agent" 的定位（请评审确认）

会议纪要中客户**全程没有提过**调优、Agent、自动化优化。客户的原话是"自动化调度与维护"，指的是**资源按需分配和自动回收**，属于 K8s 调度器的工作，不是模型层的参数或推理性能调优。这两件事名字像，工程量差一个数量级。

如果 P1 投入做 Agent：卡租不出去、账单算不对，客户第一阶段就跑不通，后面两阶段自然没有。建议这样处理：

- **P1 只做一个规则版"智能推荐"**（≈3 人天）：根据历史占用推荐空闲卡、对掉卡和高负载节点做健康度打分。不叫 Agent，能演示、能讲故事。
- **真正的调优 Agent 放到 P2**，和多机场景一起做——那时才有真实的拓扑数据和 QoS 指标可优化。
- 对客户的话术："自动化调度第一阶段就有；AI 调优要等多机跑起来才有数据，我们现在不拿 PPT 换你的预算。"

7 · 技术风险与对策

风险	影响	对策
5090 为消费级卡，无 ECC、无 MIG、驱动与企业卡不同	高	对外 SLA 按实际能力写，不承诺企业级可用性；掉卡告警必须做
2 台机器 = 平台单点，节点故障即全停	高	平台组件与卡池分离部署；如实告知，不写高可用承诺
计费与真实占用不一致（抢占、节点宕机、时钟漂移）	高	以"实例事件流"为唯一计费依据，而非定时采样；预扣 + 结算 + 每日对账三层兜底
资源回收不干净，卡被占死	中	Finalizer + 定时巡检强制回收，运维页可手动释放
对外经营算力租赁涉及资质	中	资质由客户侧解决，方案不承诺合规主体；合同口径需商务确认
单人 60 天满配排期，需求一变就崩	中	范围变更必须重算工期；每周固化一次可演示版本

8 · 需要客户确认的五个问题

1. **2 台 5090 各装几张卡？** 是 16 卡还是 8 卡——直接决定可售算力与报价模型。
2. **最小计费单位是什么？** 按小时还是按天；是否允许被抢占（允许则单价可降，但需在合同写明）。
3. **实例是否需要公网访问？** 开放 SSH / Jupyter 到公网，还是只给内网 + VPN。这决定安全投入。
4. **结算方式？** 预付余额实时扣，还是后付月结；开票主体是谁。决定计费模块的复杂度。
5. **租户是谁？** 对外部客户经营，还是集团内部 / 关联方自用。涉及资质与合规口径。

9 · 下一步

今天	本方案过评审，确认范围与 P1 边界
本周	拿到 2 台 5090 的访问权限与配置清单，跑通 GPU Operator
第 2 周	出可演示的第一版：大屏 + 手动下单一条链路
第 4 周	第一次内部 Demo，同步给客户看进度

附录 A · 工作包拆解与修正（TP0–TP9）

本表在原有拆解基础上做了四处选型修正，并把总工期压到单人可交付的范围。原拆解合计 57 人天，接近 60 天工期的 1.35 倍人力。

工作包	原方案	修正	人天
TP0 K8s 底座	直接用	2 台机器用 k3s，不调优 CNI，控制面与 GPU 共节点	3 → 2
TP1 GPU 纳管	GPU Operator	驱动装宿主机，Operator 关闭 driver 安装；5090 需 570 以上驱动	5 → 5
TP2 调度	KAI-Scheduler	**换 Volcano。**KAI 于 2026-10 才开源，生态与踩坑资料不足，60 天项目不承担试错	5 → 3
TP3 实例生命周期	二开 GPUStack GPUaaS	**不二开，改 API 对接。**GPUStack 为 Python + Vue 栈，读码改造成本高于自研同功能	8 → 5
TP4 多租户与配额	KAI Resource + K8s Quota	保留。Namespace + ResourceQuota + Volcano Queue 足够 P1	5 → 5
TP5 自助门户	GPUStack 白标 + 自研	砍购物车与算力市场，改为运营代下单 + 客户只读账单页	10 → 3
TP6 计量计费	CSGHub 参考 + 自研	**加码。**原估未覆盖支付通道、退款与对账，这是唯一不能省的部分	8 → 12
TP7 监控告警	直接用	保留，补 XID 掉卡告警与自动摘卡	4 → 4
TP8 白标打包	二开	延后，不影响客户第一轮验收	3 → 0
TP9 测试上线	自研	只做端到端 + 演示环境，客户现场部署另算	6 → 5
合计	**57 人天**	单人承诺版 **44 人天**；若投入 2 人可回到 57 人天完整范围	**44**

另外两处冗余建议直接删除：`metal3` 用于裸金属自动装机，2 台已装系统的机器上需要配 Redfish / PXE，调试成本高且无收益；`dstack` 的实例编排能力与 GPUStack 的 GPUaaS 完全重叠，二者只留一个。

附录 B · 硬件监控指标

全部由 GPU Operator 自带的 DCGM Exporter 采集，不自研采集器。RTX 5090 为消费级卡，下表中标注的字段能力需在 W1 实测确认。

指标	DCGM 字段	用途	告警阈值
GPU 利用率	`DCGM_FI_DEV_GPU_UTIL`	判定卡是否闲置，影响能否超卖	<5% 持续 30 分钟
显存已用 / 剩余	`DCGM_FI_DEV_FB_USED`	容量判断与 OOM 预警	>95% 持续 10 分钟
温度	`DCGM_FI_DEV_GPU_TEMP`	降频与宕机预警	>85°C 告警，>90°C 熔断
功耗	`DCGM_FI_DEV_POWER_USAGE`	电费成本核算	长期贴近 TDP
降频原因	`DCGM_FI_DEV_CLOCK_THROTTLE_REASONS`	性能投诉归因（温度墙 / 功耗墙）	非 0 持续 5 分钟
XID 错误	`DCGM_FI_DEV_XID_ERRORS`	**掉卡识别**，79 = 设备从总线掉落	任何非 0 立即告警并摘卡
PCIe 吞吐	`DCGM_FI_PROF_PCIE_TX/RX_BYTES`	数据加载瓶颈	长期打满
ECC 错误	`DCGM_FI_DEV_ECC_*`	—	**5090 不支持，跳过**

5090 的三个硬性事实

- `DCGM_FI_PROF_*` 类 profiling 指标在消费卡上大概率不可用，W1 需用 `dcgmproftester` 探测后再决定看板字段，避免大屏上一半是空图。
- 不支持 MIG、不支持 vGPU、没有 NVLink。整卡独占是唯一形态，多卡通信走 PCIe——这一点需在 P2 多机场景前向客户说明。
- XID 79 是消费卡高频故障。告警必须联动"把该卡标记为不可售并从池子摘除"，否则客户会租到坏卡。

附录 C · W1 落地清单

时间	动作	当天必须看到的证据
周一	宿主机装 NVIDIA 驱动（570+），两台组 k3s 集群	`nvidia-smi` 出卡；`kubectl get nodes` 两个 Ready
周二	装 GPU Operator（关闭 driver），纳管全部 GPU	allocatable 的 `nvidia.com/gpu` 合计等于总卡数
周三	DCGM + Prometheus 采集，探测可用字段	`count(DCGM_FI_DEV_GPU_UTIL)` 等于总卡数；N/A 字段清单
周四	Volcano + 租户队列与配额	超配 Pod 被 Pending，原因可读
周五	卡池地图页（每卡一格的实时状态）	能投屏演示，卡状态随实例变化

集群搭建

```
# 节点 1（控制面，与 GPU 共节点，2 台规模无需独立 master）
curl -sfL https://get.k3s.io | INSTALL_K3S_MIRROR=cn sh -s - server --disable traefik

# 节点 2（工作节点）
curl -sfL https://get.k3s.io | INSTALL_K3S_MIRROR=cn \
  K3S_URL=https://<节点1内网IP>:6443 K3S_TOKEN=<节点1的 /var/lib/rancher/k3s/server/node
```

GPU 纳管

```
# 驱动只装在宿主机，Operator 不再装驱动（避免容器内编译内核模块失败）
helm install gpu-operator nvidia/gpu-operator -n gpu-operator --create-namespace \
  --set driver.enabled=false \
```

```
  --set toolkit.enabled=true \
  --set dcgmExporter.enabled=true

# 验收
kubectl get nodes -o json | jq '.items[].status.allocatable["nvidia.com/gpu"]'
```

整卡独占的说明

`nvidia.com/gpu` 是 Kubernetes 扩展资源，只能申请整数，因此 `nvidia.com/gpu: 1` 天然就是独占一张卡，不需要任何隔离配置。Volcano 在 P1 只承担两件事：租户队列配额、排队公平；gang scheduling 留到 P2。

本方案为技术评估稿，范围与工期基于 2026-10-30 售前交流信息。客户侧资源规格、计费口径、租户性质确认后需重新校准工作量。